

Final Year Project

Balancing Quality of Experience and Energy Efficiency in Mobile Edge Computing

Luke King

A thesis submitted in part fulfilment of the degree of

BSc. (Hons.) in Computer Science

Supervisor: Professor Liam Murphy



UCD School of Computer Science

University College Dublin

April 27, 2025

Table of Contents

1	Project Specification	5
1.1	Problem Statement	5
1.2	Background	5
2	Introduction	7
3	Related Work and Ideas	8
3.1	Cloud Computing	8
3.2	Mobile Edge Computing	8
3.3	Quality of Experience	9
3.4	Edge Service Migration	9
3.5	Server utilisation Performance	10
3.6	Edge Server Energy Efficiency	10
4	Outline of Approach	12
4.1	Multi-Client Node Capabilities	12
4.2	Dynamic Latency	12
4.3	Dynamic Framework	12
4.4	New Migration Strategies	12
5	Design And Implementation	14
5.1	Addition of Multi-Client Service Nodes	16
5.2	Implementation of utilisation based latency	16
5.3	Implementation of Client satisfaction	17
5.4	Refactor of system design	18
5.5	QoE Strategy	19
5.6	Energy-Aware Strategy	21
5.7	Hybrid Strategy	22
5.8	Updated Console UI	24
6	Testing	26

6.1	Initialisation	26
6.2	Initial Placement Tests	26
6.3	Service Node Degradation	29
6.4	Improved Service Node Availability	31
6.5	Hybrid: Various Satisfaction Threshold Tests	33
6.6	Test Videos	34
7	Conclusions and Future Work	35
7.1	Reflections	35
7.2	Future Work	35

Abstract

This project extended an existing Mobile Edge Computing (MEC) framework.

The accurate placement and migration of Clients within a MEC system is crucial to ensure users are given an optimal Quality of Experience (QoE). With an increasing interest in the energy consumption of computing world-wide, it is becoming more important for MEC systems to balance the need for maintaining the users QoE whilst accounting for the systems energy efficiency.

In this project I look at the implementation of a QoE and an Energy-Aware approach in a Mobile Edge Computing Framework. Implementation of both of these approaches allowed for the addition of a third hybrid approach. This hybrid approach aimed to achieve a high level of Client QoE whilst increasing the systems energy efficiency.

Details

Source code for the MEC Framework for this project can be found at: csgitlab.ucd.ie/LKing/FYP-MEC-Framework

Source code for the Console UI for this project can be found at: csgitlab.ucd.ie/LKing/fyp-mec-fx-console-qoe

Folder containing test videos can be found at: [Google Drive Folder](#)

Chapter 1: Project Specification

Cloud computing has undergone phenomenal growth in recent years, but one of its major disadvantages is the distance between cloud servers and end-users. This problem has led to the development of Multi-Access Edge Computing (MEC) which aims to place cloud resources closer to the end-user. Previous projects developed a prototype MEC Framework, and this project will involve further development and implementation of additional features to give increased flexibility and functionality to the framework.

1.1 Problem Statement

MEC is an ever-growing field within computing. Its importance grows as the need for more powerful mobile computing increases. MEC is a step between cloud computing and a local device, allowing for computational offloading of time critical applications, or applications that simply might not have been computationally feasible on a mobile device. The implementation of MEC allows for the creation of highly mobile computing devices where most of the tasks are offloaded to nearby MEC stations. With this a plethora of problems come forward, such as, How do we provide ultra-low latency with high bandwidth reliably and consistently to mobile devices, how do we ensure that users receive adequate Quality of Experience (QoE) for any given service they would like to use, how do we ensure efficient allocation and migration of client services between MEC nodes, and how do we account for efficient energy usage across many MEC nodes within a network?

In this project I will look at a possible solution for maintaining client Quality of Experience (QoE) across an MEC network by means of smart allocation and migration, whilst keeping the system aware of energy consumption across the network.

1.2 Background

“Mobile Edge Computing (MEC) is a new network architecture concept that provides IT and cloud computing capabilities at the edge of the mobile network. As in close proximity to subscribers, MEC can offer a service environment with ultra-low latency, high-bandwidth, and direct access to real-time network information.” [1].

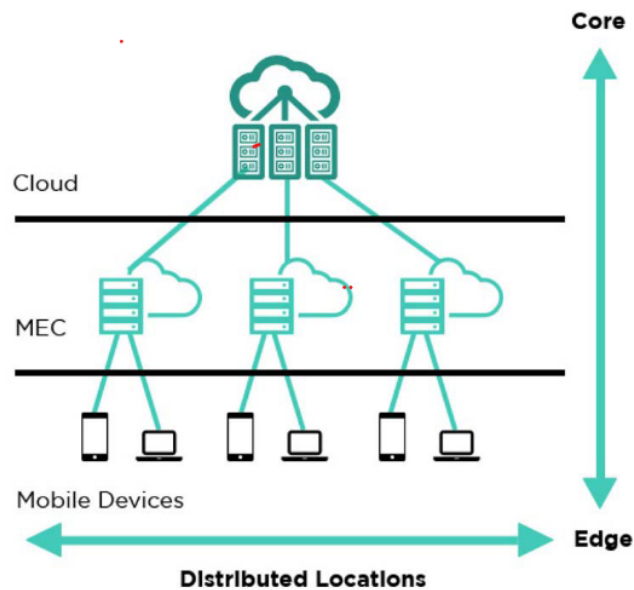


Figure 1.1: MEC Diagram [2]

MEC is a network between cloud computing and local devices, allowing for easier computational offloading for time-critical tasks where offloading to the Cloud would not have sufficed. Being between cloud computing and local computing, it is an extension of cloud computing, this allows for it to have the many benefits that standard cloud computing has, including scalability, energy efficiency, cost reduction, etc. However the extent of each benefit is often less than that compared to broader cloud computing.

The key advantages of using MEC compared to standard cloud computing includes [2]:

Reduced Latency, due to the proximity of MEC to mobile devices, they can provide an ultra-low latency experience. This also lends to high-bandwidth as mobile devices no longer need to offload to a much further away Cloud data centre, instead offloading to a much closer MEC service, avoiding the low-bandwidth restrictions of long-range transmission.

Enhanced User Experience, the ultra-low latency combined with the high-bandwidth allows for enhanced user quality of experience (QoE), being that the high-bandwidth allows for compute hungry applications such as augmented reality (AR), video analytics, etc. The low latency allows for high quality streaming, gaming, etc.

Energy Efficiency, as mobile devices can offload their computational workload to MEC services, the large amount of energy required for such tasks can be saved on the mobile device, allowing for longer battery life. Minimizing this energy usage is a big use case for MEC as it allows for mobile devices to have the claimed all day battery life.

Chapter 2: Introduction

Following review of past projects work within the field, I decided to explore modification of the existing Mobile Edge Computing (MEC) Framework worked on by James Needham, Jerry Klos, Joes Dempsey and Darragh Clarke. This framework is a functional prototype simulating an MEC network consisting of an Orchestrator, Service Nodes, and Clients. A Service Node is a server capable of hosting a Client and its application, while the Orchestrator is the overseer of both the Service Nodes and the Clients, managing initial allocation of Clients, as well as the migration of Clients when necessary.

The current framework assigns a static constant latency to Service Nodes upon creation, meaning a Service Nodes created with 100ms (milliseconds) of latency, will always have 100ms of latency. It is my intention to make this latency be calculated dynamically. As the latency a Client perceives from a Service Node is dependant on multiple forms of latency, being local computational latency, remote computational latency, communication latency and file request latency [3], my intention is to base the new dynamic Service Nodes latency off of primarily computational latency, allowing the Service Node to calculate its latency using utilisation.

Whilst the system does work well, it is relatively simplistic, allowing for only a one-to-one relationship between Clients and Service Nodes, being that a Service Node can only ever host a single Client. This severely limits the flexibility of the framework, as most real MEC systems allow for a many-to-one relationship between Clients and Service Nodes. One goal for this project is to update this and allow a Service Node to host many Clients, in turn, this will also improve the effect of a load-based latency approach as a Service Nodes load will be much more variable. This change will require re-working how the framework utilises Service Nodes and allocate Clients to Service Nodes, as well as updating how migrations are managed across Service Nodes.

New Orchestrator placement and migration strategies are also to be added, one prioritising placement and migration of Clients on Service Nodes to maximise each Clients over all QoE score (satisfaction). Accounting for possible changes to the new dynamic latency as the number of Clients on a Service Node change, varying the utilisation. Another strategy will aim to make the Orchestrator energy-aware, allocating and migrating Clients on the basis of reducing the overall energy consumption of the system. The overall goal is to make a third strategy, combining both previously mentioned strategies into a single hybrid strategy. This strategy should be capable of balancing Client satisfaction and system energy usage.

Chapter 3: Related Work and Ideas

In this chapter, I will outline the research I have performed of topics relating to this project within cloud computing and edge computing, as well as current research in managing a clients satisfaction (QoE) and power usage within servers and MEC.

3.1 Cloud Computing

According to NIST [\[4\]](#) cloud computing is defined as,

"a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction."

Cloud computing allows for Clients (users) to offload their computational, storage, or service needs to a distant computer within a data centre. This lets the Clients avoid the large upfront cost of hardware and infrastructure whilst also improving scalability and flexibility due to the large amount of compute power and ease of access.

3.2 Mobile Edge Computing

As previously mentioned within section [1.2](#), Mobile Edge Computing (MEC) is an extension of cloud computing with the design of bringing increased compute power closer to users, allowing for ultra-low latency and increased user quality of service. The key characteristics of MEC are [\[5\]](#):

1. On-Premises: MEC has access to its own local resources, allowing it to run isolated from the network. Meaning that MEC can perform its own compute without requiring to offload to another remote service for all its tasks. This is important when dealing with security for systems that require high levels of resilience
2. Proximity: MEC is deployed at the edge of a network, meaning it should be one jump away from mobile devices. This gives it the advantage to analyse big data in an ultra-low latency environment.
3. Location Awareness: as MEC is managed at the edge and is part of a wireless network, MEC can use low-level signalling to determine the location of each connected mobile device. This allows for entirely new use cases of computing, making use of location based services, analytics, etc.

3.3 Quality of Experience

Quality of Experience (QoE) is determined as a measurement of user factors such as perceived satisfaction or feeling of enjoyment of a user for an application or service [6]. Due to the nature of Quality of Experience (QoE) being based on a users perceived satisfaction or enjoyment, it can be highly subjective and change from one user to another, this can make it hard to quantify with exact metrics. Previously Quality of Service (QoS) was used by vendors within Service Level Agreements (SLA) to assure users of a guaranteed level of service provided. However, as users needs and perceived satisfaction evolve over time, QoE emerged as a more important metric for maintaining service satisfaction. Most metrics used within QoS can be used within QoE as what is determined to provide a service at a given level of satisfaction is often true across most users of a service. These metrics can be shown as what a certain application or service needs to be capable of running properly, such as bandwidth, CPU cores available, latency, etc.

As discussed within [7], the major difference between QoE and QoS is that the critical metric is very dependent on the application or service. For example, online gaming or time-sensitive services such as heart monitoring devices require consistent low latency, e.g. for online gaming < 100ms (milliseconds) of delivered latency. Delivered latency consists of the round-trip latency between a Client and Server, meaning it is both the communication latency and the computation latency combined. As such ensuring a very low communication latency and computation latency are key to ensuring a delivered ultra-low latency.

Another common metric used for QoS is CPU frequency and/or CPU core count as-well as RAM when the computational tasks being done require heavy computing resource [8]. Ensuring the required amount of processing power is available for applications allows for achieving a higher level of QoS and thus QoE. However, as [9] shows, simply ensuring that each application/service receives its required resource is not enough, because as the total utilisation of an edge resource is increased the computational latency increases, thus increasing the total delivered latency the end-user experiences.

3.4 Edge Service Migration

A service migration is the act of moving a service or application from one server node to another. There are many types of migrations capable within MEC and cloud computing. One of the main reasons to migrate a service is a drop in QoS, services will have a set level of requirements for operating, be that latency, CPU frequency/core count, RAM, bandwidth, etc. If these requirements drop below the required minimum, a service migration might be triggered. The destination of a migration can vary from, another nearby MEC server node that can fulfil the services requirements, to offloading the service to the cloud, where Latency will increase but the computational resources are vastly superior [10].

There are two main types of migration, those are a dynamic migration or a manual migration. Dynamic migrations are triggered dynamically, in this frameworks case, an Orchestrator is checking for Client's satisfaction reports and if they are deemed unsatisfied, a migration is dynamically triggered. Manual migration is one that is triggered manually and often is not as seamless as a dynamic migration, often interrupting the service.

A service migration should be a stateful migration of a service or application, being that, when a migration is triggered, the services internal state data is saved upon stopping of the service, and transferred with the service to the new Node. Upon migration the service can continue from

exactly where it stopped prior to the migration.

3.5 Server utilisation Performance

As was previously mentioned in 3.3, the computational performance of a server or node varies depending on the total utilisation of the node. Understanding the effect this has on computational performance and computational latency is vital to ensuring a balanced use of MEC nodes, allowing for high level of QoE for users on those nodes. The paper [9] performs edge performance benchmarks on multiple different deployments to assess the effect of edge resource stress on different deployments computational and communication latencies. The study found that while the communication and computational latencies vary across different deployments and that the communication latency of applications can improve by using a cloud-edge deployment, there can be no overall gain if the computational latency is high on resource restricted edge nodes.

The application I will be looking at most from this is RealFD as it is defined as being latency critical and computationally intensive, which suits the framework as it will be modelled off of node core availability and delivered node latency. The results showed that for using just the edge nodes for computation, as the resource stress increased, the computational latency increased greatly. The given chart is provided below:

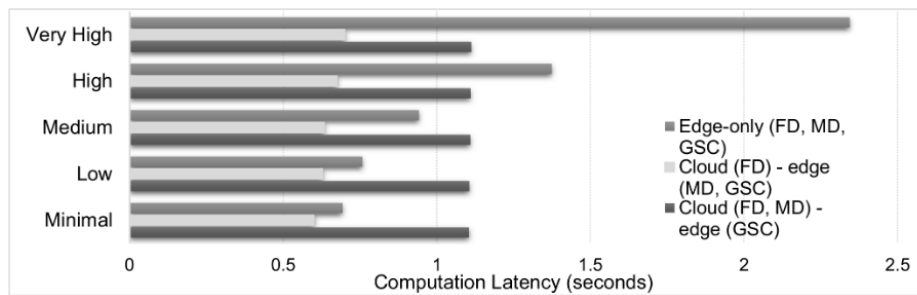


Figure 3.1: Computational latencies of RealFD when the edge resource is stressed [9]

3.6 Edge Server Energy Efficiency

Energy consumption of servers and datacentres is a big topic in the field of cloud computing, as the number of Cloud datacentres and MEC node facilities increase, the need to monitor the energy consumption grows. A single edge servers energy consumption is low, however as the quantity of edge servers grows to provide adequate coverage of large cities, the total energy consumption starts to become enormous [11].

The energy consumption of a server is dependent on many things, primarily its CPU, memory, disk and network card. Out of these the main contributor is the CPU, as such, most server energy consumption models are mostly represented by its CPU utilisation [12]. Looking at how to model the CPU utilisation to energy consumption it was found that due to the fact that a processors power consumption is directly related to the number of transistors used, a linear relationship can be formed between a CPU's utilisation and its energy consumption [13, 14].

It is also found that a significant percent of a server's peak power is consumed whilst simply

idling, up to 66% [14]. With both facts combined it is clear that the best way to efficiently save energy across edge servers is to simply prioritize turning off idle servers and attempt to improve the utilisation of servers already running. As can be seen from the evaluation of [12], they determine that if the utilisation of servers are higher within a virtualized datacentre, then fewer servers must be activated to host a service workload, and thus the energy consumption of the virtualized datacentre is reduced. In the case for this project, all MEC nodes combined would be considered the virtualised datacentre.

Chapter 4: Outline of Approach

4.1 Multi-Client Node Capabilities

One of the future additions James Needham [7] discussed was the improvement of the frameworks multi-Client capabilities. One of my goals is to enable a Service Node to host multiple Client applications. This, in turn, allows for dynamic latency calculation based on system load, resulting in a more complex system and more insightful tests.

4.2 Dynamic Latency

A key upgrade to the framework, in terms of making it more dynamic, is refactoring how Service Nodes calculate the latency that they deliver to their Clients. Instead of using a static latency assigned at creation, a Service Node's latency will now be calculated based on its utilisation. As discussed in section 3.5, an increase in utilisation (load) leads to increased computational latency. In turn, this increases the latency delivered from the Service Node to the Client. Each Client application will have a defined core load, increasing the utilisation of the hosting Service Node. Combined with Multi-Client capable Service Nodes, the updated framework will allow for much more complex migration scenarios based on load.

4.3 Dynamic Framework

Currently, the Orchestrator runs most of the framework, checking the state of the Service Nodes and Clients at periodic intervals via a defined heartbeat. I intend to refactor this into a more dynamic, event-based system. In the updated approach, both Service Nodes and Clients will notify the Orchestrator using WebSocket messages when specific events occur, such as application start/stop, changes in Node delivered latency, or changes in Client satisfaction. This will significantly speed up the responsiveness of the system, as it will no longer have to wait for the next heartbeat to detect when a migration is required. Instead, Clients and Service Nodes will provide the Orchestrator with real-time updates.

4.4 New Migration Strategies

Three new migration and placement strategies will be added to the Orchestrator. The first will be a QoE only approach where the objective is to give each Client the best possible delivered latency from a Service Node. This will utilize a round-robin approach, striping Clients across active Service

Nodes. This will reduce load on each Service Node which will minimize the delivered latency, thus maximizing the satisfaction for Clients. A satisfaction threshold will be given within the orchestrator (80%). Upon a Client's satisfaction dropping below this defined threshold, the Orchestrator will attempt to migrate to a better Service Node.

Secondly an Energy-Aware only approach will be added, where the objective will be to reduce the consumption of the system as a whole. As was discussed within section 3.6, up to 66% of a server's peak power is consumed whilst idle, leading to the evaluation that increasing the utilisation of servers in a system, and thus reducing number of servers needing to be active, increases energy efficiency. Understanding this shows that an Energy-Aware strategy is in-fact primarily the opposite of the previously mentioned QoE only approach. Instead of striping Clients in a round-robin style, load each Service Node up to an allowed maximum, and only then begin loading the next Service Node. In this framework, a Service Node with zero utilisation can be considered de-activated, and thus improving energy efficiency.

The final strategy to be implemented will be a hybrid of the two previously stated. This Energy-Aware QoE approach will attempt to improve energy efficiency using the same method as the Energy-Aware only approach, but the Orchestrator will also account for a Clients satisfaction. With a lowered satisfaction threshold compared to of the QoE only approach (60%), this strategy will load a Service Node up until either its maximum allowed utilisation, or until hosted Clients begin dropping below the satisfaction threshold. Upon dropping below the satisfaction threshold, Clients will be migrated to the next Service Node, repeating the process of loading again.

Chapter 5: Design And Implementation

This chapter will detail the design changes to James Needham's MEC Framework and explain how the updates are implemented as follows:

- Addition of multi-Client capabilities to Service Node.
- Implementation of utilisation-based latency to Service Node.
- Implementation of Client QoE satisfaction score.
- Refactor the system design to become dynamic and event based.
- QoE and Energy-Aware migration strategies added to the Orchestrator.
- Hybrid Energy-Aware QoE migration strategy added to the Orchestrator.
- Refactor Framework Console UI.

This is a project expansion and refactor of the previously created MEC Framework. The project will keep many of the original components previously implemented, while adding new functionality as-well as modifying existing functionality. The project is programmed in Java 11 utilising Maven to handle library management and compilation. The Java WebSocket library will be utilised to handle message passing across all three distinct modules.

Below, the diagram [5.1](#) is an illustration of the updated Framework. Key changes that can be observed include; a JarManager to handle multiple Client applications; ServiceNode notifies the Orchestrator of its utilization and latency on both heartbeat and in the event of an application start/stop; Orchestrator forwards the received latency to the relevant Clients, in turn, the Clients reply with their updated Satisfaction.

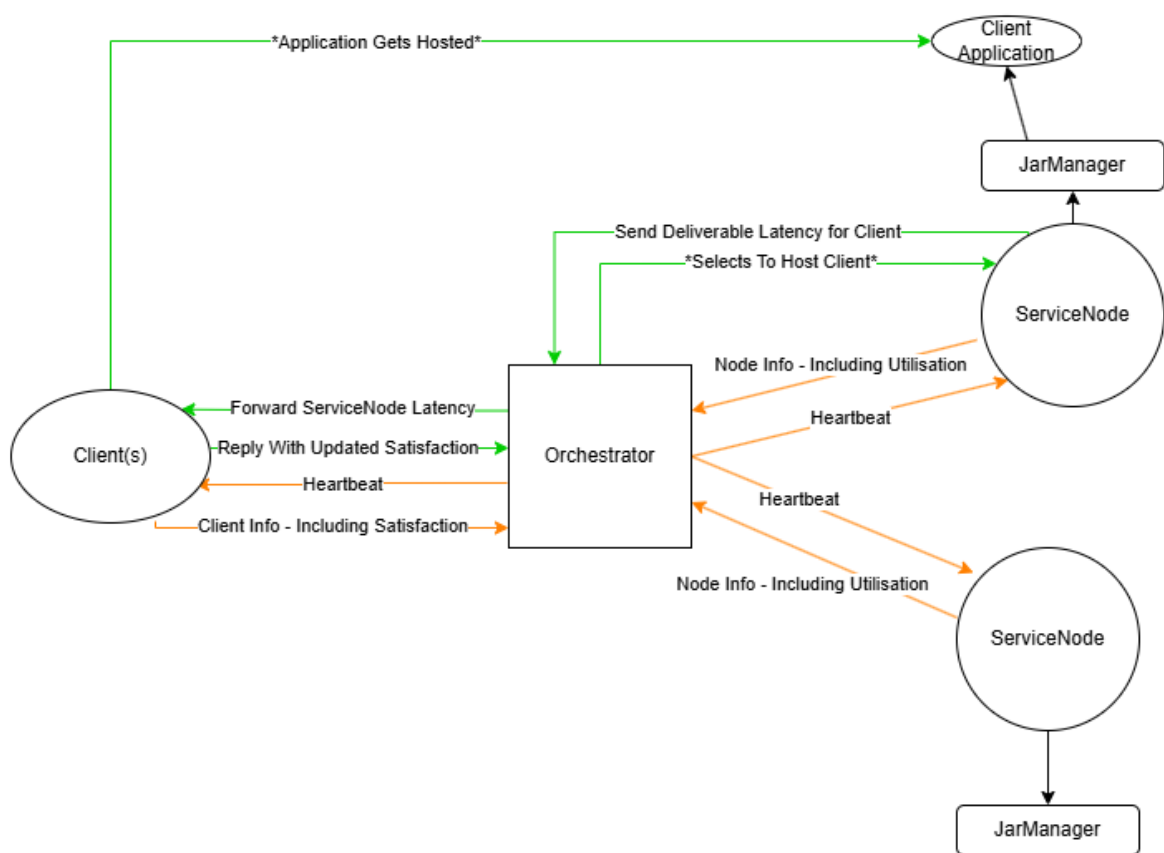


Figure 5.1: Updated MEC Framework Implementation

5.1 Addition of Multi-Client Service Nodes

The first extension to the framework is the addition of multi-Client capabilities to Service Nodes. Initially, each Service Node contained a single JarController class. This class is responsible for starting/stopping and managing the jar application provided from a Client.

A new class has been implemented, ServiceControllerManager, this class is a manager class for multiple instances of JarController. The manager handles starting/stopping and all previous functions an individual JarController has, but across multiple controllers, utilising a UUID to determine which application. When a Service Node receives a start or stop application request from the Orchestrator, the message contains the Client's UUID, allowing for the ControllerManager to map the Client's application to the UUID.

To reflect this new change within the system, the relationship map within Orchestrator that previously mapped a Service Node to a Client has been flipped, now mapping a Client to a Service Node. This change allows for a Service Node to be related to multiple Clients, when this was not possible with the previous version.

Finally, the addition of a Client-CPU map has been added to Service Node. Upon starting and stopping of a Client's application, the CPU core requirement defined for that application is added to and removed from this map, respectively. This allows for each individual Service Node to keep track of both its total CPU core load, as-well as each individual applications CPU core load.

5.2 Implementation of utilisation based latency

Previously both Client and Service Node did have defined CPU core counts, however these were only used if a Client had specified CPU cores as its QoE metric. The Client's satisfaction was then calculated as: the Clients required CPU cores as a percentage of the Service Nodes CPU cores. This option has been removed as the effect Client CPU load has on a Service Node, primarily translates to computational latency as discussed within section 3.5. This can then be considered within the delivered latency. As a result, a Client's QoE no longer can be based on CPU core counts, instead it will be only latency.

Service Node now dynamically calculates its latency. This calculation is a step function based on the Node's CPU core utilisation. The starting latency increases in steps as the utilisation increases. As was shown in section 3.5, The relationship between utilisation and latency is not linear, as such, as the Service Node's utilisation increases, the delivered latency increases exponentially.

Upon any new Client application starting/stopping, the Service Node updates its CPU core utilisation and then subsequently updates its deliverable latency. The calculation for deliverable latency is displayed as a chart below in figure 5.2.

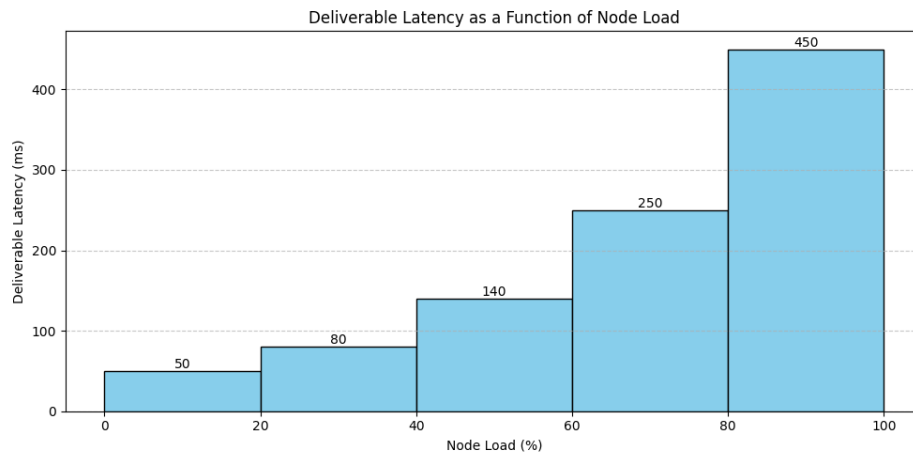


Figure 5.2: Service Node Latency Utilisation Chart

The decision to increment steps in 20% intervals was made to align with the figure 3.1, where computational latency was broken into 5 bins, ranging from minimal usage to very high usage. The assigned latency values allow for a Service Node to host multiple Clients (with base demand of 100ms), while also quickly increasing in latency past 40%.

5.3 Implementation of Client satisfaction

Clients' satisfaction has been implemented in a latency-based approach. Each Client is created with a defined required latency, this is the latency the Client wants to receive from a Service Node in order to be considered 100% satisfied. The default required latency is 100ms.

The formula used to calculate a Client's satisfaction is as follows; The Clients required latency as a percentage of its received latency, with a maximum of 100. Given a Client with a requirement of 100ms, this results in a satisfaction of 100% for any received latency below and equal to 100ms; Client's satisfaction then declines following the graph below.



Figure 5.3: Client Satisfaction Graph - 100ms required latency

Above shows the degradation in Client satisfaction as received latency increases. Satisfaction drops sharply initially after exceeding the Clients required latency, gradually flattening as latency increases. This translates to a Client having a greater rate of loss in satisfaction initially after exceeding the desired latency, with the rate of loss in satisfaction decreasing as received latency further increases.

5.4 Refactor of system design

Updates to the flow of the Framework have made the system more dynamic and real-time. Previously, the Orchestrator would periodically examine each Service Node in a round-robin fashion to determine if its hosted Client would get better satisfaction on a different Service Node, migrating if such conditions were true. The Orchestrator also calculated the Client's satisfaction internally and notified the Client of its satisfaction.

The existing WebSocket message, ClientSatisfactionAlert, has been refactored; It is no longer sent from the Orchestrator to the Client, updating the Client. The Client now sends a satisfaction alert to the Orchestrator upon any update of its satisfaction, informing the Orchestrator of the Client's new satisfaction. This new approach better represents an MEC system, as only a Client can determine its own QoE satisfaction, as well as speeds up the system as the Orchestrator is no longer re-calculating Client satisfaction at periodic intervals, instead receiving a real-time update upon change.

Client re-calculates its satisfaction whenever it receives a new latency alert. These latency alerts are facilitated by the new WebSocket message, ServiceNodeLatencyAlert. These alerts are generated within Service Node, upon any change to CPU utilisation, Service Node re-calculates its deliverable latency and dispatches a latency alert for each Client hosted by that Service Node. This alert contains the deliverable latency as well as the UUID of the intended Client recipient. The alerts are sent to the Orchestrator, which then forwards each alert to the Client, using the given UUID within the alert.

This new latency alert combined with the new approach for Client satisfaction alert allows for real-time updating of a Service Nodes latency and, in turn, notifying and updating of hosted Client's satisfaction. This provides a smooth flow of assigning a Service Node to host a Client. The Service Node updates it's utilisation and latency based on the Client's CPU requirements. This new latency is sent to all hosted Clients. Each Client updates it's satisfaction and notifies Orchestrator. This all happens dynamically without the need for the Orchestrator to periodically check on either Client or Service Node.

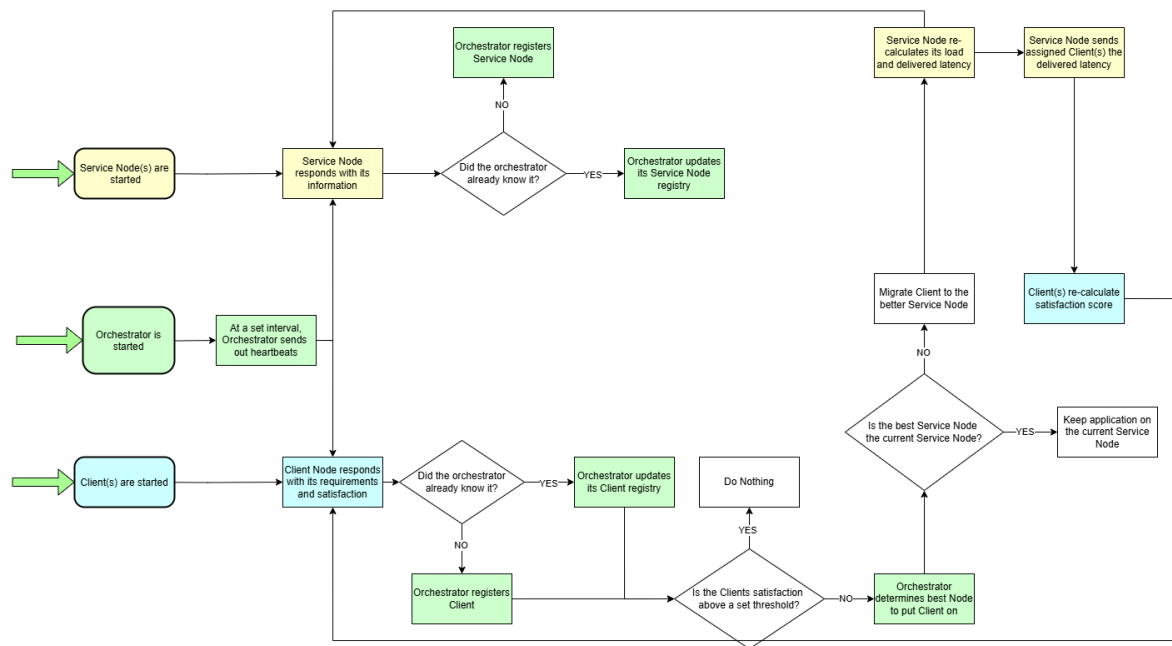


Figure 5.4: Updated Framework Flowchart

Above is a flowchart depicting the new design of the Framework. Actions handled by Service Node are shaded in yellow, actions handled by Orchestrator are shaded in blue, and actions handled by Client are shaded in blue.

While there is still a heartbeat function within the Orchestrator, this is to ensure all Clients and Service Nodes remain active and other information remains synced. The previous Orchestrator approach of periodically inspecting Service Nodes in a round-robin fashion has been removed. Clients are now placed on a Service Node upon registration and only considered for migration if their satisfaction is below the defined threshold for that migration strategy, or in the case of Energy-Aware only, if a Service Node is detected as being degraded.

5.5 QoE Strategy

The first strategy to be implemented is the QoE only strategy. Previously the algorithm determined the best Service Node by calculating the Client's satisfaction and selecting the Service Node that provides the maximum satisfaction. Now, the the best Service Node is determined as the Service Node with the lowest latency and not exceeding 100% utilisation. This algorithm is used when a new Client is registered for placement, and when a Client satisfaction alert reaches the Orchestrator below the defined satisfaction threshold.

Algorithm 1 QoE Only Approach

```
clientCPURequirement  $\leftarrow$  Client.getCpuRequirement()
bestNode  $\leftarrow$  null
bestSimulatedLatency  $\leftarrow$  Float.MAX_VALUE

if nodes contains no elements then
    return bestNode
end if

if client has a hosting ServiceNode then
    hostingNodeUUID  $\leftarrow$  clientServiceNodeMap.getHostingNode
end if

for all ServiceNode  $\in$  nodes do

    if ServiceNode.UUID = hostingNodeUUID then

        if ServiceNode.currentLatency  $\leq$  bestSimulatedLatency and ServiceNode.load
 $\leq$  100 then
            bestNode  $\leftarrow$  ServiceNode
            bestSimulatedLatency  $\leftarrow$  ServiceNode.currentLatency
        end if

    else

        currentLatency  $\leftarrow$  ServiceNode.estimatedLatency(clientCPURequirement)
        if currentLatency < bestSimulatedLatency and ServiceNode.estimateLoad  $\leq$ 
100 then
            bestNode  $\leftarrow$  ServiceNode
            bestSimulatedLatency  $\leftarrow$  ServiceNode.currentLatency
        end if
    end if
end for
return bestNode
```

The above pseudocode describes the algorithm for placement and migration within the QoE only strategy. This method takes in two parameters, the Client to be possibly migrated and a collection of ServiceNodes called nodes. The output of this is used within the Orchestrator to determine if a migration will take place. If the returned Service Node is the hosting Node, then no migration takes place; If the returned Service Node is a different Node, then the Client will be migrated to that more optimal Service Node. In the event of no Service Node being returned, the Orchestrator will place the Client on a placeholder Cloud Node, simulating referring the Client to the cloud beyond the MEC.

A key part of the selection algorithm is to account for a change in Service Node utilisation. Due to Service Node latency being based on utilisation, placing a Client on to a Service Node could lead to an increase in latency, negating the benefit of the migration. This leads to a Client hopping between Service Nodes; As a Client is migrated from one Service Node to another, the previous Service Node can improve in latency, with the new Service Node declining in latency. This then leads to another migration, migrating the Client back to the previous Service Node, allowing for the cycle to continue.

To account for this issue, Service Node has been given a method to simulate a latency. This method takes in a CPU core count and returns the latency if that CPU core count were added to the Service Node, this allows for the Orchestrator to test what a Service Node's latency will become when adding a Client, before actually placing the Client on that Service Node. This allows the algorithm to avoid placing a Client on a Service Node that will have an increase in latency making it equal or worse than the current Service Node. The last part to make this selection work, is ensuring to handle the current hosting Service Node differently. If the hosting Service Node were to be checked the same as all other Service Nodes, the CPU core count from the Client will be counted twice, as such, a separate if statement is utilised to handle the hosting Service Nodes current latency and not estimated latency. An estimateLoad method was also added to Service Node to do a similar function as estimateLatency, but instead of returning a simulated latency, it returns a simulated load.

5.6 Energy-Aware Strategy

The Energy-Aware only strategy effectively does the opposite of QoE only, with only an interest in reducing the total number of Service Nodes being used. As there is no interest in maintaining a level of satisfaction across Clients, the algorithm used does not need to account for any levels of latency or satisfaction, only availability on Service Nodes.

As Client satisfaction is of no concern, Service Nodes are not selected when a Client sends the Orchestrator a satisfaction alert. Instead, Service Nodes are selected only on initial placement of a Client, and in the event of a Service Node becoming degraded; A degraded Service Node is a Node that has a reduced CPU core count in this Framework, going from a base of 10 cores to 4 cores. This leads to a spike in utilisation percentage, thus, increasing latency greatly. The Orchestrator handles degraded Service Nodes by attempting to migrate all currently hosted Clients on that Service Node to another Service Node.

Algorithm 2 Energy-Aware Only Approach

```

cpuRequirement ← Client.getCpuRequirement()
bestNode ← null

if nodes contains no elements then
    return bestNode
end if

for all ServiceNode ∈ nodes do

    if ServiceNode.estimatedLoad(cpuRequirement) ≤ 85 then
        if bestNode = null or ServiceNode.load > bestNode.load then
            bestNode ← ServiceNode
            bestSimulatedLatency ← ServiceNode.currentLatency
        end if
    end if

end for
return bestNode

```

Above is the algorithm used for determining the optimal Service Node for placement and migration

of a Client in the Energy-Aware only strategy. There are two key parts within this algorithm, the first is to estimate the load on the Service Node, the second is selecting a Service Node with the highest load. Similar to the QoE approach, the Orchestrator must get an estimate of the Service Node's utilisation if the Client's application were to be hosted. This allows correct limiting of the max utilisation allowed on a Service Node, in this case 85%. Selecting the Service Node with the highest utilisation (load) ensures the Orchestrator correctly fills Service Nodes before moving to the next Service Node, doing this allows for us to minimise the total amount of Service Nodes being utilised across the system as Clients are added. It is also important to check if the bestNode is null before comparing utilisation as this allows for initial selection of a bestNode.

Similar to the QoE approach, if the algorithm returns no Service Node, signifying all Service Nodes are at maximum utilisation, the Orchestrator places the Client on a placeholder Cloud Node. This simulates all available Service Nodes being full to capacity, resulting in a Client being referred back to the Cloud behind the MEC system.

5.7 Hybrid Strategy

The third and final strategy to be added is the hybrid strategy. This strategy is a balance between a QoE approach and an Energy-Aware approach. With the intention of increasing energy efficiency using the same methods as the Energy-Aware approach while still accounting for a Client's satisfaction. The core of this implementation is to utilise the Energy-Aware placement method, loading up Service Nodes before moving to a second Service Node, while using latency-based satisfaction scores to determine when and where to migrate a Client to.

As this strategy determines the need for migrations based on satisfaction, the selection process is done any time the Orchestrator receives a satisfaction alert from a Client, in the same fashion as the QoE approach. The key difference being a lowered satisfaction threshold within the Orchestrator, from 80% to 60%. This allows for a more balanced approach of migrations, allowing Clients to become somewhat unsatisfied until they reach below 60% satisfaction. At that point, a migration is triggered, attempting to move the Client to a Service Node with better service. A higher satisfaction threshold will lead to less Clients on a Service Node as the Orchestrator will enforce a higher Client satisfaction level. Conversely, a lower satisfaction threshold will allow more Clients on a Service Node as the Orchestrator allows Client satisfaction to be lower, allowing more utilisation on Service Nodes.

A key part of the selection logic is sorting the collection of Service Nodes given to the algorithm. All given Service Nodes are sorted in descending order of utilisation, this allows the algorithm to iterate through Service Nodes starting at the most utilised Service Node. This is important as it is needed for correct Service Node selection when considering a migration from an existing Service Node. Instead of iterating through all Service Nodes finding the most optimal one, like QoE, this algorithm iterates through Service Nodes with decreasing utilisation until reaching a Service Node giving a better level of satisfaction, the algorithm then stops and does not continue through the rest of the Service Nodes.

Algorithm 3 Energy-Aware QoE Approach

```
cpuRequirement  $\leftarrow$  Client.getCpuRequirement()
bestNode  $\leftarrow$  null
currentSatisfaction  $\leftarrow$  0
bestSatisfaction  $\leftarrow$  0

if nodes contains no elements then
    return null
end if

sortedServiceNodes  $\leftarrow$  nodes sorted by descending node load

if client has a hosting ServiceNode then

    hostingNodeUUID  $\leftarrow$  clientServiceNodeMap.getHostingNode
    bestSatisfaction  $\leftarrow$  Client.SimulatedSatisfaction(hostingNode.Latency)  $\triangleright$  set the
    bestSatisfaction as the Clients satisfaction given the current hosts latency

    for all serviceNode  $\in$  sortedServiceNodes do
        if serviceNode.uuid = hostingNodeUUID then
            currentSatisfaction  $\leftarrow$  Client.SimulatedSatisfaction(serviceNode.Latency)
            if currentSatisfaction  $\geq$  bestSatisfaction then
                bestSatisfaction  $\leftarrow$  currentSatisfaction
                bestNode  $\leftarrow$  serviceNode
            end if
        else
            simulatedLatency  $\leftarrow$  serviceNode.estimatedLatency(cpuRequirement)
            currentSatisfaction  $\leftarrow$  Client.SimulatedSatisfaction(simulatedLatency)
            if currentSatisfaction > bestSatisfaction and
            serviceNode.estimatedLoad(cpuRequirement)  $\leq$  85 then
                bestSatisfaction  $\leftarrow$  currentSatisfaction
                bestNode  $\leftarrow$  serviceNode
                return bestNode
            end if
        end if
    end for

else

    for all serviceNode  $\in$  sortedServiceNodes do

        if serviceNode.estimatedLoad(cpuRequirement)  $\leq$  85 then
            if bestNode = null or serviceNode.load > bestNode.load then
                bestNode  $\leftarrow$  serviceNode
                bestSimulatedLatency  $\leftarrow$  serviceNode.currentLatency
            end if
        end if

    end for
end if

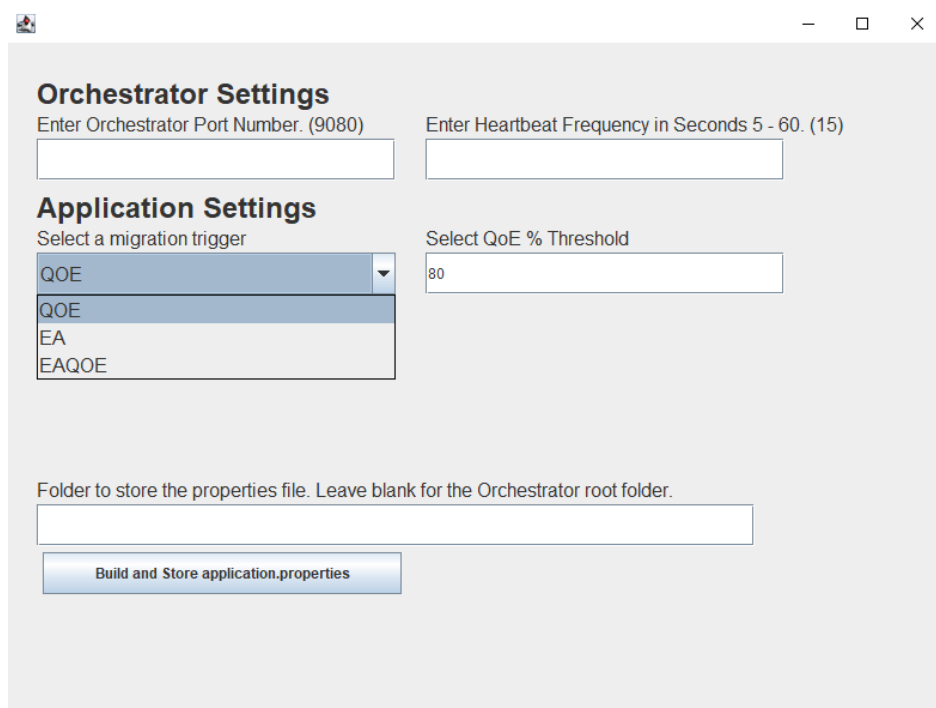
return bestNode
```

Above is the algorithm for the Energy-Aware QoE selection strategy. The algorithm is split into two main parts using an initial check to see if the Client has a current hosting Service Node. This check is to determine if the algorithm is used for initial placement or for a migration from an existing Service Node. In the event of initial placement, the Energy-Aware logic is utilised, prioritising placing Clients on a most loaded Service Node. This will increase the energy efficiency of the system by reducing the total number of Service Nodes required initially.

When there is a current hosting Service Node the algorithm works similar to the QoE approach. However, instead of just selecting an optimal Service Node based on latency; the Client's satisfaction is used. A method was added to the Client for simulating satisfaction, this method takes in a given latency and returns a satisfaction score. The satisfaction score returned is the satisfaction the Client would have if it were to be placed on a Service Node with that latency. Combined with the methods for simulating latency within Service Node, the Orchestrator can simulate what happens when a Client is placed on a Service Node and the satisfaction of a Client with the updated latency, all before committing to the placement. Unlike the QoE approach, this algorithm will exit when a Service Node different than the host is found. This is because if it does not return the Service Node then, it will continue iterating through remaining Service Nodes with decreasing utilisation, thus guaranteeing they will be selected. This return ensures the algorithm only returns the next best Service Node from the hosting Service Node, as opposed to the absolute best Service Node.

5.8 Updated Console UI

To follow with the updated framework, the accompanying console UI project has also been updated. The Orchestrator settings panel has been refactored to better reflect the available settings within the system. The new selection and migration strategies have been added in place of the previous options. A text box appears when selecting either QoE or Energy-Aware QoE (EAQOE) allowing the user to enter a custom satisfaction threshold for the entered strategy.



The screenshot shows a window titled "Orchestrator Settings" with a standard Windows title bar (minimize, maximize, close buttons). The window is divided into two main sections: "Orchestrator Settings" and "Application Settings".

Orchestrator Settings

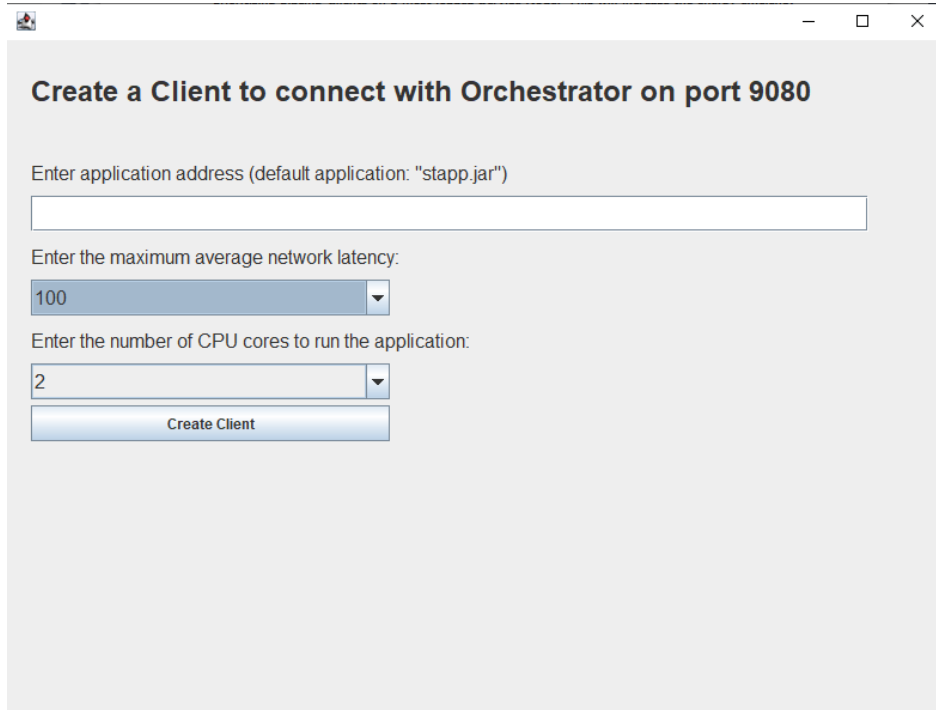
- Enter Orchestrator Port Number. (9080): A text input field.
- Enter Heartbeat Frequency in Seconds 5 - 60. (15): A text input field.

Application Settings

- Select a migration trigger: A dropdown menu with options: QOE, QOE, EA, and EAQOE. The first "QOE" option is currently selected.
- Select QoE % Threshold: A text input field containing the value "80".
- Folder to store the properties file. Leave blank for the Orchestrator root folder.: A text input field.
- Build and Store application.properties: A button.

Figure 5.5: Updated Orchestrator Settings Panel

The Client creation panel has also been updated to reflect the new Client updates. The Client panel now allows inputting both latency requirement and CPU requirements at the same time. The available latencies have also been changed to better reflect the latency values within the system. Both values are passed on creation of a Client, latency is used within the Client to calculate satisfaction, and CPU requirement is used for providing utilisation on a Service Node.



The screenshot shows a window titled "Create a Client to connect with Orchestrator on port 9080". Inside the window, there are three input fields and a button. The first field is for the application address, with a hint "(default application: 'stapp.jar')". The second field is for the maximum average network latency, with a dropdown menu showing "100". The third field is for the number of CPU cores, with a dropdown menu showing "2". Below these fields is a button labeled "Create Client".

Figure 5.6: Updated Client Creation Panel

Chapter 6: Testing

All three new strategies will be assessed within the MEC Framework. Each strategy will be tested across various test scenarios. Below are the test scenarios that will be considered for testing:

- Initial Placement Testing
- Migrations upon Service Node degradation
- Migrations upon new Service Node(s) becomes available

After all three strategies have been assessed on the previous scenarios, further tests will be conducted on the Hybrid strategy, exploring the effect of different satisfaction thresholds with regards to Client placement and migration frequency.

6.1 Initialisation

The MEC Framework can be initialised using the accompanying Framework Console UI. The Console UI allows for editing settings and starting the Orchestrator, configuring and starting Service Nodes, and configuring and starting Clients. Upon starting the Orchestrator, a Service Node and Client can also be started. Upon starting of either a Service Node or Client, Orchestrator will send an initialisation message to the respective service. A heartbeat between Orchestrator and any Service Nodes and Clients can be observed, showing the system is functioning. The system runs on the hosts device. To ensure the Orchestrator functions correctly, the port the Orchestrator runs on must be available (9080).

6.2 Initial Placement Tests

Initial placement tests involved the creation of Service Nodes, followed by Clients, observing the behaviour of the system with regard to the placement of Clients on Service Nodes and if any migrations take place. The environment for each test is as follows:

Three Service Nodes are started in the default configuration of 10 CPU cores and a starting latency of 50ms. The increase in latency with Service Node utilisation can be found in figure 5.2. Three Clients are then created with a requirement of 100ms of latency while consuming 2 CPU cores.

6.2.1 QoE Test

On registration, the first Client was placed on the first Service Node. This led to the first Service Node rising to 20% utilisation, which, in turn, increased latency to 80ms. With a Client demand

of 100ms, a Service Node latency of 80ms provided the Client with 100% satisfaction. The next Client to be registered was placed on the next empty Service Node. This Service Node received the same utilisation as the first Service Node; thus, the second Client was provided with 100% satisfaction. The final Client was placed on the last empty Service Node, and following the first and second Client, was provided with 100% satisfaction from that Service Node.

After all Clients had been placed, the final system had an average Client satisfaction of 100%, with each Client being placed on its own Service Node. The system also had all 3 Service Nodes under load, with each Service Node having 20% utilisation.

This result is as intended for the QoE approach. Each Client was placed across the Service Nodes in a round-robin manner, maximising satisfaction while disregarding energy efficiency.

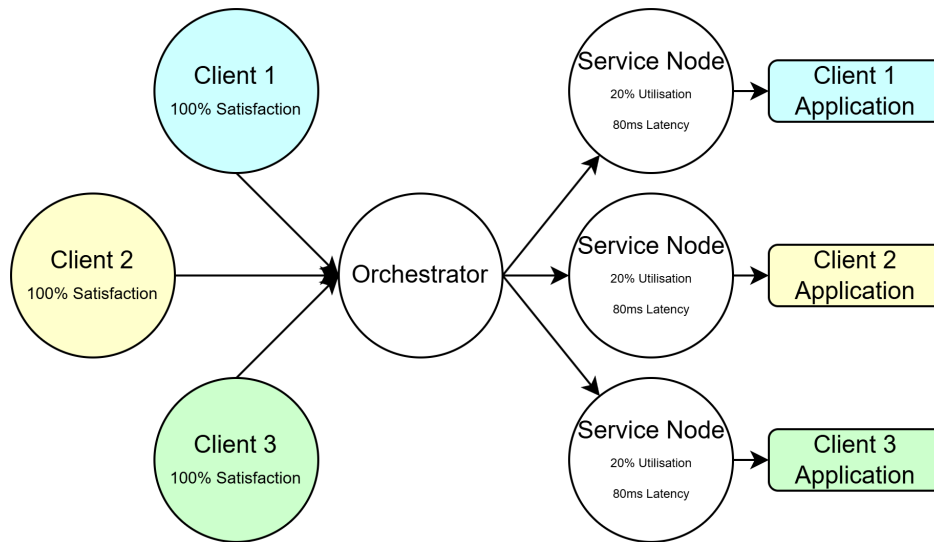


Figure 6.1: QoE Placement Testing Results

6.2.2 Energy-Aware Test

On registration, the first Client was placed on the first Service Node. This led to the first Service Node rising to 20% utilisation, which, in turn, increased latency to 80ms. With a Client demand of 100ms, a Service Node latency of 80ms provided the Client with 100% satisfaction. The next Client was placed on the same Service Node as before. The Service Node's utilisation became 40%, creating latency to 140ms. This increase in latency caused both the new Client and the previous Client to fall to 71.43% satisfaction. The final Client was placed on the same Service Node as the previous two. The Service Node's utilisation became 60%, delivering a latency of 250ms. This further increase led to all 3 Client's satisfaction scores to drop to 40%.

After all Clients had been placed, the final system had an average Client satisfaction of 40%, with all Clients being placed on the singular Service Node. The system only loaded one Service Node, up to a utilisation of 60%. All other Service Nodes remained at 0% utilisation. In this framework we can consider these two Service Nodes turned off, resulting in higher energy efficiency.

This result is as intended for the Energy-Aware approach. All Clients loaded on a single Service Node until not possible, minimising active Service Nodes while disregarding Client satisfaction.

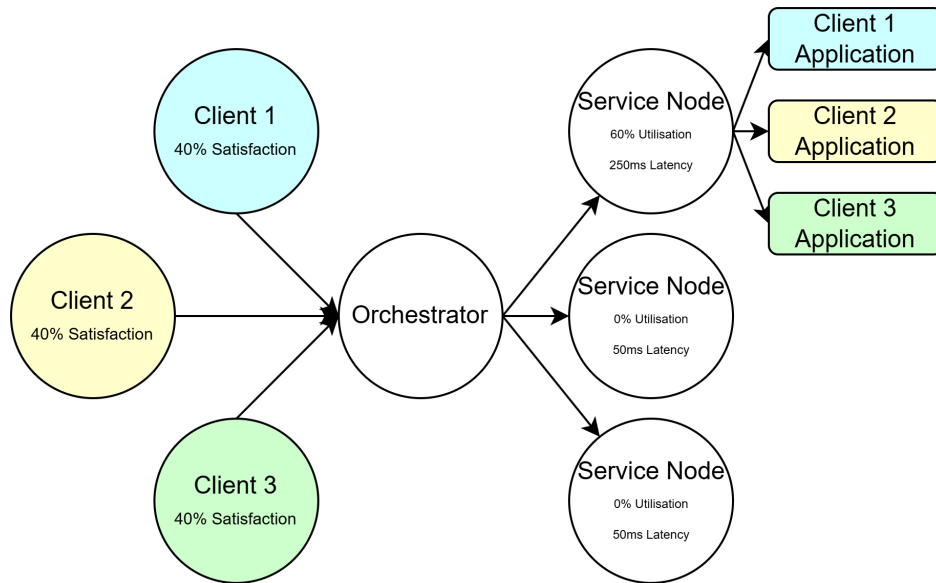


Figure 6.2: Energy-Aware Placement Testing Results

6.2.3 Hybrid Test

On registration, the first Client was placed on the first Service Node. This led to the first Service Node rising to 20% utilisation, which, in turn, increased latency to 80ms. With a Client demand of 100ms, a Service Node latency of 80ms provided the Client with 100% satisfaction. The next Client was placed on the same Service Node as before. The Service Node's utilisation became 40%, creating a latency of 140ms. This increase in latency caused both the new Client and the previous Client to fall to 71.43% satisfaction. The Orchestrator attempted to place the final Client on the same Service Node as the previous two Clients, however Client satisfaction would fall below the defined satisfaction threshold (60%). The Orchestrator detected this and migrated a Client from the first Service Node to a second empty one. This resulted in 2 Clients on the first Service Node as before with 71.43% satisfaction, and a Client on a second Service Node with a satisfaction of 100%.

After all Clients had been placed, the final system had an average Client satisfaction of 80.95%, with two Clients on one Service Node and a singular Client on a second Service Node. The system loaded two Service Nodes, the first at 40% and the second at 20%. The third Service Node remained at 0% utilisation. The third Service Node can be considered turned off, providing increased energy efficiency.

This result is as intended, the strategy provided a balance between both QoE and Energy-Aware strategies. Average Client satisfaction fell between QoE's 100% and Energy-Aware's 40%; The number of active Service Nodes also fell between QoE's three active Service Nodes and Energy-Aware's one active Service Node. It is important to note that, as the Orchestrator does not account for Client or Service Node tenure-ship, that being how long a Client has been on a Service Node, and for the fact that all Clients are homogenous. The migration of the Client to the second Service Node will not always be the third Client, a previous Client could be migrated, with the new Client taking its place on the first Service Node.

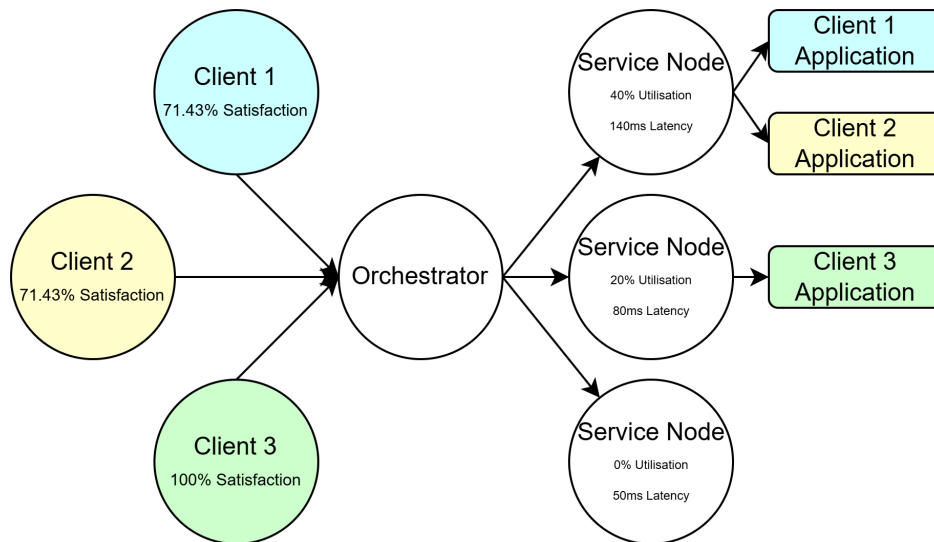


Figure 6.3: Hybrid Placement Testing Results

6.3 Service Node Degradation

Service Node Degradation tests involved creating a scenario following the initial placement. However, a single Service Node degrades in quality after a set interval, resulting in the Service Node decreasing in total CPU core availability, allowing for quicker over-loading and increase in latency. The environment for each test is as follows:

One Service Node is started in default configuration, but is set to degrade after 30 seconds, reducing from 10 CPU cores to 4 CPU cores. Two more Service Nodes are also started in default configuration with 10 CPU cores. Three Clients are then created with a requirement of 100ms of latency while consuming 2 CPU cores. This will result in the initial placement results. Once 30 seconds have passed from creation of the degrading Service Node, the Node will reduce to 4 CPU cores. The tests will observe how the system manages this. The test will ensure that the Service Node to degrade will be the Service Node with the most utilisation.

6.3.1 QoE Test

Following the Client placements from section 6.2.1; the Service Node to degrade can be any Service Node as they are in equal states. Upon the Service Node degrading, the CPU core count reduced to 4 CPU cores. This increased utilisation from 20% to 50% usage, leading to a delivered latency of 140ms. The increase in latency reduced the hosted Client's satisfaction to 71.43%. The Orchestrator did not decide to migrate the affected Client, remaining with a satisfaction of 71.43%.

This behaviour is correct as any migration within the system will lead to further reduced average Client satisfaction, due to the fact that moving the Client to another Service Node will reduce two Clients to 71.43% satisfaction. This results in the system's average Client satisfaction being 90.47%, with the number of active Service Nodes remaining the same at zero.

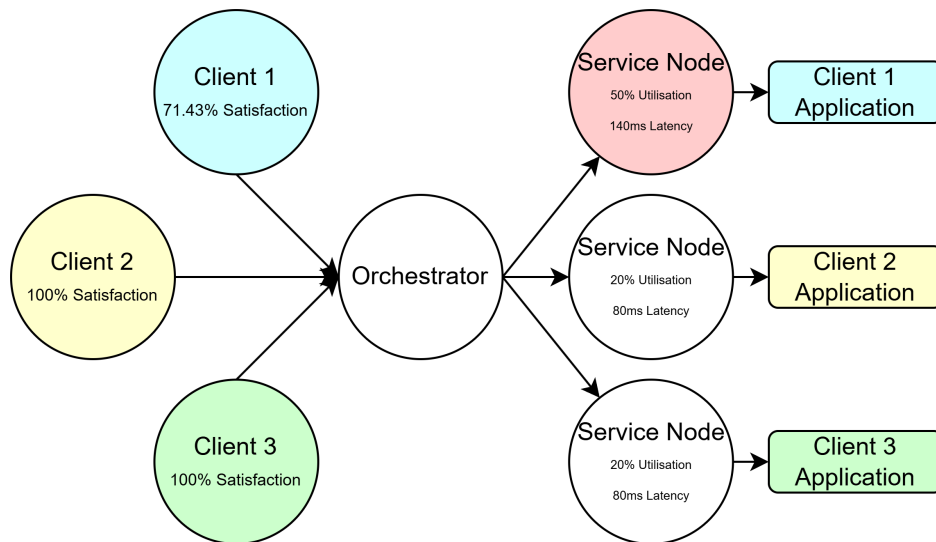


Figure 6.4: QoE Degradation Testing Results

6.3.2 Energy-Aware Test

Following the Client placements from section 6.2.2; the Service Node to degrade is the Service Node with all three Clients hosted on it. Upon the Service Node degrading, the CPU core count reduced to 4 CPU cores. This increased utilization from 60% to 150%, overloading the Service Node beyond 100%, resulting in the maximum latency of 450ms, reducing the satisfaction of the three Clients to 22.22%. As the Service Node is overloaded the Orchestrator triggered a migration, migrating all Clients off the degraded Service Node. All three Clients were then migrated to a single new Service Node, raising their satisfaction back to the previous 40%.

This behaviour is correct as it retained the same energy efficiency as before degradation, with two Service Nodes at 0% utilisation, while not allowing any Service Node to become overloaded. The average Client satisfaction, as well as the number of active Service Nodes remained the same as initial placement testing.

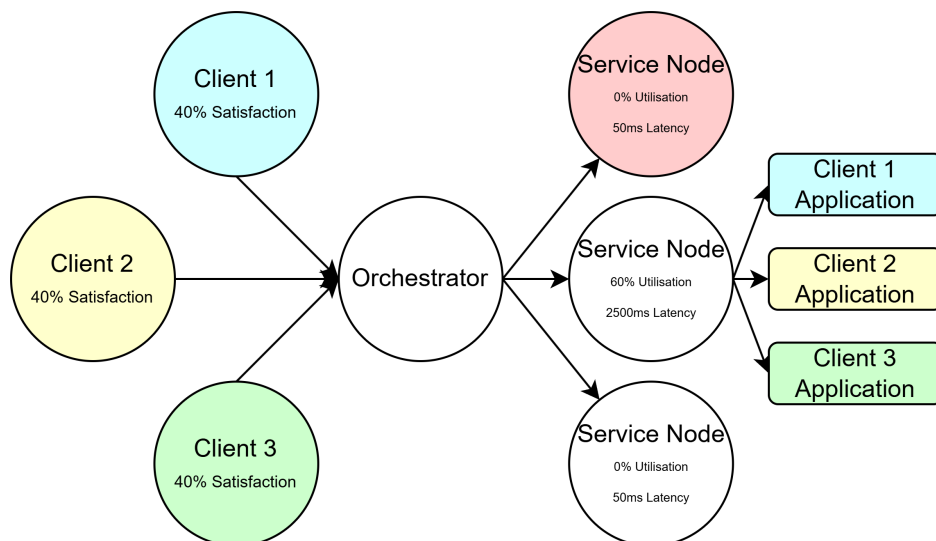


Figure 6.5: Energy-Aware Degradation Testing Results

6.3.3 Hybrid Test

Following the Client placements from section 6.2.3; the Service Node to degrade is the Service Node with two Clients hosted on it. Upon the Service Node degrading, the CPU core count reduced to 4 CPU cores. This increased utilisation from 40% to 100%, resulting in the maximum latency of 450ms, reducing the satisfaction of the two hosted Clients to 22.22%. As the Clients are below the satisfaction threshold of 60%, the Orchestrator attempted to migrate. The Orchestrator migrated a single Client from the degraded Service Node to the Service Node hosting the third Client. The effect of this migration lowered the degraded Service Nodes utilisation to 50%, providing the Client with a satisfaction of 71.43%. The Service Node with two Clients increased in utilisation to 40%, providing both Clients with a satisfaction of 71.43%.

This behaviour is correct as all Clients were above the satisfaction threshold and a Service Node was still at 0% utilisation, improving energy efficiency. The average Client satisfaction dropped from 80.95% to 71.43%, however as all Clients were above the threshold, the Orchestrator will not perform any more migrations.

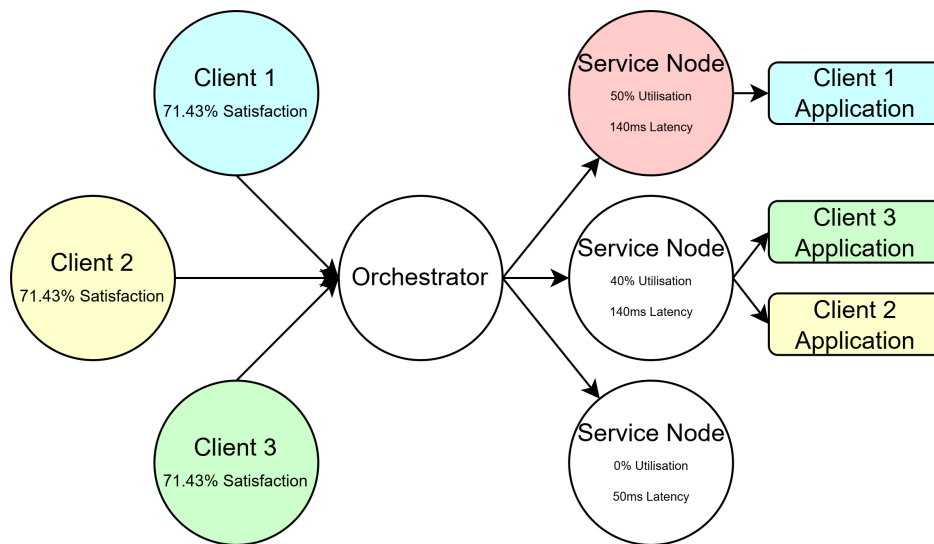


Figure 6.6: Hybrid Degradation Testing Results

6.4 Improved Service Node Availability

The improved Service Node availability tests involved a scenario in which new Service Node(s) are added to the system after the initial placement of Clients. These tests observe how each Strategy handles the migrations of Clients. The environment for each test is as follows:

One Service Node is started in default configuration with 10 CPU cores. Three Clients are then created with a requirement of 100ms while consuming 2 CPU cores. This will result in all three Clients being placed on the single Service Node. Two new Service Nodes with 10 CPU cores will then be created, allowing new Service Nodes for the Three already hosted Clients to migrate to.

6.4.1 QoE Test

Starting with all three Clients on a single Service Node, all Clients had a satisfaction of 40%, with a Service Node utilisation of 60%. Upon creation of a second Service Node, the Orchestrator migrated a Client from the initial Service Node to the newly created Service Node. The result improved satisfaction for all Clients. The migrated Client increased the new Service Nodes utilisation to 20%, receiving a satisfaction of 100%. The initial Service Node reduced to 40% utilisation, providing an improved satisfaction of 71.43% to the remaining two Clients. Creation of a third Service Node yielded a similar result, the Orchestrator migrated a Client from the initial Service Node to the new third Service Node, further improving the satisfaction for both Clients. The migrated Client increased the new Service Node to 20%, receiving a satisfaction of 100%. The migration reduced the utilisation of the initial Service Node to 20%, providing 100% satisfaction to the remaining Client.

The result is as intended, replicating the initial placement from Figure 6.1, where all three Service Nodes were created before creation of the Clients. The Orchestrator detected possible migrations on creation of a Service Node and correctly migrated only one Client to improve average Client satisfaction. The final average Client satisfaction was 100%, with all three Service Nodes becoming utilised.

6.4.2 Energy-Aware Test

Starting with all three Clients on a single Service Node, all Clients had a satisfaction of 40%, with a Service Node utilisation of 60%. Upon creation of a second Service Node, the Orchestrator did not attempt any migration, leaving the three Clients on the initial Service Node. The Orchestrator performed the same on creation of the third Service Node, keeping all Clients on the initial Service Node.

This result is as intended, replicating the initial placement from figure 6.2, where all three Service Nodes were created before creation of the Clients. The Orchestrator correctly did not migrate upon creation of new Service Nodes as the currently loaded Service Node is still available, thus reducing the total number of Service Nodes being utilised. The final average Client satisfaction was 40%, with only one Service Node being utilised.

6.4.3 Hybrid Test

Starting with all three Clients on a single Service Node, all Clients had a satisfaction of 40%, with a Service Node utilisation of 60%. Upon creation of a second Service Node, the Orchestrator migrated a Client from the initial Service Node to the newly created Service Node, following the performance of the QoE approach. The migration resulted in an improvement in satisfaction for all Clients, with the migrated Client receiving 100% satisfaction on a new Service Node, and the two remaining Clients improve to 71.43% satisfaction. With all Clients being above the satisfaction threshold of 60%, the Orchestrator decided to not migrate any Clients, leaving the new Service Node at 0% utilisation

This result is as intended, replicating the initial placement from figure 6.3, where all three Service Nodes were created before creation of the Clients. The Orchestrator migrated the first Client, improving all Clients satisfaction and correctly did not migrate the second Client having as all Clients were above the satisfaction threshold, preserving energy efficiency. The final average Client satisfaction was 80.95%, with two Service Nodes being utilised.

6.5 Hybrid: Various Satisfaction Threshold Tests

The satisfaction threshold tests involved further testing using the hybrid Orchestrator strategy, altering the satisfaction threshold and then performing placement tests. The tests observe how the Orchestrator manages placement and migration of Clients when an alternative satisfaction threshold is provided.

Two tests were conducted: One with a satisfaction threshold of 80%, another with a satisfaction threshold of 40%. These tests document how the Orchestrator handles both stricter and more lenient Client migrations. For the environment setup refer to section [6.2](#).

6.5.1 80% Threshold Test

On registration, the first Client was placed on the first Service Node; raising the Service Nodes utilisation to 20% and receiving a latency of 80ms, providing a satisfaction of 100%. Upon registration of the second Client, the Orchestrator attempted to place the second Client on the same Service Node as the first Client, resulting in an increase in utilisation to 40%, dropping both Clients satisfaction to 71.43%; As this is below the newly defined satisfaction threshold of 80%, the Orchestrator migrated a Client from the first Service Node to a second Service Node, providing both Clients with 100% satisfaction. Registration of the third Client showed similar results. The Orchestrator attempted to place the new Client on the first Service Node, however Client satisfaction rose to 71.43%, resulting in the Orchestrator migrating a Client to a third Service Node. The end result was one Client on each Service Node with a satisfaction of 100%. All three Service Nodes were being utilised with a utilisation of 20% each.

This result is as intended, showing that the Orchestrator can correctly maintain Clients above an increased satisfaction threshold, resulting in stricter and more frequent migrations within the system. The test also illustrates that as satisfaction threshold is increased, Client satisfaction becomes more important in determining Client placement, in turn, making the hybrid approach perform closer to the QoE only approach. This test showed identical results to the QoE only approach, shown in figure [6.1](#).

6.5.2 40% Threshold Test

On registration, the first Client was placed on the first Service Node; Raising the Service Nodes utilisation to 20% and receiving a latency of 80ms, providing a satisfaction of 100%. Upon registration of the second Client, the Orchestrator placed the second Client onto the same first Service Node; increasing utilisation to 40%, resulting in both Clients satisfaction decreasing to 71.43%. No migration was triggered. Registration of the third Client showed similar results. The Orchestrator placed the third Client on the same Service Node as the previous two Clients, increasing utilisation to 60% and decreasing all Clients satisfaction to 40%. No migration was triggered. The end result was all three Clients on a single Service Node, all with a satisfaction of 40%. Two Service Nodes were not being utilised, as such, they can be assumed to be turned off, increasing energy efficiency.

This result is as intended, showing that the Orchestrator correctly allows a lower overall Client satisfaction when given a decreased satisfaction threshold, resulting in more lenient and less frequent migrations within the system. The test also illustrates that as satisfaction threshold is decreased, Client satisfaction becomes less important in determining Client placement, with the Orchestrator prioritising energy efficiency instead; Making the hybrid approach perform closer to the Energy-

Aware only approach. This test showed identical results to the Energy-Aware only approach, shown in figure 6.2.

6.6 Test Videos

Test case	Result	Video
6.2.1 QoE Placement Test	Correct	link
6.2.2 Energy-Aware Placement Test	Correct	link
6.2.3 Hybrid Placement Test	Correct	link

Table 6.1: Placement Tests: Test Cases, Results and Videos

Test case	Result	Video
6.3.1 QoE Degradation Test	Correct	link
6.3.2 Energy-Aware Degradation Test	Correct	link
6.3.3 Hybrid Degradation Test	Correct	link

Table 6.2: Degradation Tests: Test Cases, Results and Videos

Test case	Result	Video
6.4.1 QoE Improved Availability Test	Correct	link
6.4.2 Energy-Aware Improved Availability Test	Correct	link
6.4.3 Hybrid Improved Availability Test	Correct	link

Table 6.3: Improved Availability Tests: Test Cases, Results and Videos

Test case	Result	Video
6.5.1 80% Satisfaction Threshold Test	Correct	link
6.5.2 40% Satisfaction Threshold Test	Correct	link

Table 6.4: Hybrid Satisfaction Threshold Tests: Test Cases, Results and Videos

Chapter 7: Conclusions and Future Work

7.1 Reflections

This project aimed to improve an existing Mobile Edge Computing (MEC) Framework by implementing new initial placement and migration strategies, taking into account a Client's Quality of Experience (QoE) and the systems overall energy efficiency. The implementation of this goal was facilitated by, the addition of dynamic latency calculations within each Service Node based on utilisation; updated calculations of each Client's QoE satisfaction score; a rework of the frameworks interactions between Orchestrator, Clients, and Service Nodes. Thus, creating a more dynamic and real-time framework. The addition of multi-Client capabilities to Service Nodes, allowing for more complex and larger scale environments; the addition of three new placement and migration strategies, with primary interest in a hybrid approach balancing a Client's QoE score with the frameworks energy efficiency, allowing for varying levels of Client satisfaction and energy efficient placement. Updates to a companion UI Console project were also achieved to better reflect launching and using the updated MEC Framework.

The goal of this project was successfully achieved, with the Framework demonstrating correct initial placement and migrations across all three strategies, as can be seen within chapter 6. Videos demonstrating these correct tests can be found within section 6.6. The modification of the systems interactions as-well as the addition of multi-Client capabilities combined with utilisation-based latency calculations provides a strong extension of the Framework, opening the door to future changes with a more dynamic and real-time Framework.

7.2 Future Work

While the improvements to James Needham's MEC framework are significant, there are still multiple additions that can be considered, leaving room for future work for someone to continue the development of the Framework. Below are some possible ideas for future development.

7.2.1 Multiple factors for Client QoE

Currently a Client's satisfaction score is calculated solely on latency, however there are multiple other factors to a Client's QoE that can be considered. Some of these factors are, network jitter, packet loss, bandwidth, etc.

Exploring a way of combining multiple of these factors into a singular Client QoE score could improve the flexibility of the framework, allowing Service Nodes to model not just latency, but other network and hardware related factors. This could also allow the Orchestrator to prioritise some specific types of satisfaction, perhaps prioritising bandwidth over latency depending on the specific application.

7.2.2 Variation within Service Node configurations

Service Nodes currently can be configured to have varying amounts of CPU cores, and also be set to degrade/improve in CPU core count. This brings a degree of heterogeneity, allowing stronger and weaker Service Nodes to be present within the Framework. Further scenarios tests involving varying Service Nodes would be of benefit for exploring how each strategy handles more complex environments.

Additional degree's of heterogeneity could further improve the depth of Service Nodes, such as the addition of different latency calculations, providing alternate latency outputs for a given level of utilisation

7.2.3 Variation within Client

A Client can be configured with different demands in latency as well as different requirements of CPU cores to run the application, further expansion of this to include not just changing the latency demand, but altering the formula used to calculate a Clients satisfaction could allow for more complex scenarios to be tested. Currently the Client follows a satisfaction formula described in section [5.3](#), the formula follows a non-linear relationship, making it hard for a Client to quickly reach 0% satisfaction. A new linear formula could be added, providing a hard limit, in which, the Client will reach 0% satisfaction.

7.2.4 Increased Scaling

While the scale of tests within this project has increased, testing three Clients across three Service Nodes. Further testing with increased scale could provide more in-depth insight into how each strategy performs when exposed to a much larger scale, for example 15+ Clients with 10+ Service Nodes.

Acknowledgements

I would like to thank Prof. Liam Murphy for his continuous support and guidance through the course of this project, helping me whenever I was unsure of how to continue within the project. I would also like to thank James Needham for sharing his previous work and making it available for me to use. Finally, a special thank you to my mum for all her support and encouragement throughout the year.

Bibliography

1. Li, H., Shou, G., Hu, Y. & Guo, Z. *Mobile Edge Computing: Progress and Challenges in 2016 4th IEEE International Conference on Mobile Cloud Computing, Services, and Engineering (MobileCloud)* (Mar. 2016), 83–84. https://ieeexplore.ieee.org/abstract/document/7474412?casa_token=ZAR0pjIhY7IAAAAA:rxoQ3RPrjvPuSq2cnX5w6-x93JfcfIMOMv-u2BwtvMFUYxNO5EnqME5ofjK_Mewmru1E_-sQ.
2. Abbas, N., Zhang, Y., Taherkordi, A. & Skeie, T. Mobile Edge Computing: A Survey. *IEEE Internet of Things Journal* **5**. Conference Name: IEEE Internet of Things Journal, 450–465. ISSN: 2327-4662. https://ieeexplore.ieee.org/abstract/document/8030322?casa_token=sP5svv-qOoAAAAA:R66Y_3bloexlXmJmyK63yUDDT_SmS2MyOTPPc19S1M72gBIpEsWDUzecnVohWwA (Feb. 2018).
3. Chen, C.-L., Brinton, C. G. & Aggarwal, V. Latency Minimization for Mobile Edge Computing Networks. *IEEE Transactions on Mobile Computing* **22**. Conference Name: IEEE Transactions on Mobile Computing, 2233–2247. ISSN: 1558-0660. <https://ieeexplore.ieee.org/abstract/document/9557844> (Apr. 2023).
4. Mell, P. & Grance, T. The NIST Definition of Cloud Computing. en.
5. Patel, M. *et al.* Mobile-Edge Computing – Introductory Technical White Paper. en.
6. Laghari, A. A., He, H., Khan, A., Kumar, N. & Kharel, R. Quality of Experience Framework for Cloud Computing (QoC). *IEEE Access* **6**. Conference Name: IEEE Access, 64876–64890. ISSN: 2169-3536. <https://ieeexplore.ieee.org/abstract/document/8438876> (2018).
7. Needham, J. Quality of Experience based Migration in Edge Computing. en (2024).
8. Lai, P. *et al.* *Quality of Experience-Aware User Allocation in Edge Computing Systems: A Potential Game in 2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS)* ISSN: 2575-8411 (Nov. 2020), 223–233. https://ieeexplore.ieee.org/abstract/document/9355646?casa_token=aSBVxwVqKnsAAAAA:jF8ep8qjr5ipRv5SsOQ6WLeIGu9526EsAi0R8.
9. McChesney, J., Wang, N., Tanwer, A., de Lara, E. & Varghese, B. *DeFog: fog computing benchmarks in Proceedings of the 4th ACM/IEEE Symposium on Edge Computing* (Association for Computing Machinery, New York, NY, USA, Nov. 2019), 47–58. ISBN: 978-1-4503-6733-2. <https://dl.acm.org/doi/10.1145/3318216.3363299>.
10. Wang, S., Xu, J., Zhang, N. & Liu, Y. A Survey on Service Migration in Mobile Edge Computing. *IEEE Access* **6**. Conference Name: IEEE Access, 23511–23528. ISSN: 2169-3536. <https://ieeexplore.ieee.org/abstract/document/8340768> (2018).
11. Li, Y. & Wang, S. *An Energy-Aware Edge Server Placement Algorithm in Mobile Edge Computing in 2018 IEEE International Conference on Edge Computing (EDGE)* (July 2018), 66–73. <https://ieeexplore.ieee.org/abstract/document/8473378>.
12. Wang, S., Liu, Z., Zheng, Z., Sun, Q. & Yang, F. *Particle Swarm Optimization for Energy-Aware Virtual Machine Placement Optimization in Virtualized Data Centers in 2013 International Conference on Parallel and Distributed Systems* ISSN: 1521-9097 (Dec. 2013), 102–109. https://ieeexplore.ieee.org/abstract/document/6808163?casa_token=7mMtguZWihYAAAAA:1CXg9yJr6FB-gqfQ5LvKSrQtaYlplAV_1YX10ATfjVvikVKJRi1Cq4U7TWyn10-WQrQn7Pk1.
13. Gupta, V., Nathuji, R. & Schwan, K. An analysis of power reduction in datacenters using heterogeneous chip multiprocessors. *SIGMETRICS Perform. Eval. Rev.* **39**, 87–91. ISSN: 0163-5999. <https://dl.acm.org/doi/10.1145/2160803.2160867> (Dec. 2011).

-
14. Chen, G. *et al.* Energy-Aware Server Provisioning and Load Dispatching for Connection-Intensive Internet Services. en.